Agentic Systems Patterns

A practical reference for modern agent architecture: goals, loops, tools, skills, memory, protocols, multi-agent systems, and production runtimes.

INTRODUCTION

Agentic Systems Patterns

This book is a practical reference for designing, testing, and operating agentic systems.

The catalog favors patterns that help engineers build real systems: control loops, goals, state, tools, skills, protocols, memory, multi-agent coordination, runtime observability, and evaluation.

Runnable examples live in the repository root. Each chapter links back to the relevant source folder when code is available.

Publishing

The public site is designed for GitHub Pages at:

```
https://gturitto.github.io/Agentic-Systems-Patterns/
```

Offline release artifacts live in `book/releases`.

The generated PDF is published at:

```
https://gturitto.github.io/Agentic-Systems-Patterns/releases/Agentic-Systems-Patterns.pdf
```

License

Single Agent

A single agent receives a goal or message, consults its context, and produces an answer or action. This is the smallest useful unit in the catalog.

Use it when one model-backed worker can complete the task without delegation, durable orchestration, or protocol-level interoperability.

Avoid it when the task needs stateful retries, external approvals, multiple specialists, or independent evaluation.

Source: `single-agent-pattern`

Agent Loop

The agent loop turns a model call into an agent: observe, decide, act, evaluate, and stop.

```
flowchart LR
  G[Goal] --> O[Observe]
  O --> D[Decide]
  D --> A[Act]
  A --> E[Evaluate]
  E -->|continue| O
  E -->|done| R[Result]
```

The core engineering work is bounding the loop. Define max iterations, cost, timeout, cancellation, and success criteria before the loop starts.

Source: `agent-loop-pattern`

Goals and State

Goals define success. State records progress.

Agents become easier to resume, debug, and evaluate when the goal is stored separately from chat history. A useful state record tracks the current objective, constraints, completed steps, pending work, evidence, approvals, and errors.

Source: `goals-and-state-pattern`

Tool Use

Tool use gives an agent a controlled interface to external capability: calculators, search, databases, code execution, files, APIs, or business systems.

The tool boundary should be typed, validated, observable, and permissioned. Tool descriptions are part of the agent-computer interface, not decoration.

Source: `tool-using-agent-pattern`

Structured Output

Structured output constrains model responses to typed data that software can validate and consume.

Use schemas for routing decisions, tool plans, policy outcomes, database writes, and workflow transitions. Validate every response before using it.

Source: `structured-output-pattern`

Context Engineering

Context engineering controls what the model sees: instructions, state, retrieval results, tool documentation, memory, examples, and prior messages.

Good context is selective. It gives the model enough information to act while protecting the context window from noisy transcripts and stale data.

Source: `context-engineering-pattern`

Planning and Execution

Planning separates deciding what to do from doing it. The planner creates steps; the executor runs them, reports progress, and handles errors.

Use this when the task has meaningful sequencing, dependencies, or recoverable failure points.

Source: `planning-pattern`

ReAct

ReAct alternates reasoning and acting. The agent reasons about the current state, calls a tool or takes an action, observes the result, then repeats.

ReAct is useful for tool-rich tasks where the agent cannot know all required information upfront.

Source: `react-pattern-reason-act`

Reflection

Reflection asks an agent or evaluator to inspect prior output and identify improvements. It is most useful when paired with concrete criteria.

Avoid reflection loops that only produce longer output. Reflection should change decisions, tests, state, or final artifacts.

Source: `reflection-and-self-improvement-pattern`

Evaluator-Optimizer

Evaluator-Optimizer pairs a generator with an evaluator. The generator proposes; the evaluator scores; the optimizer revises or stops.

```
flowchart LR
  I[Input] --> G[Generate]
  G --> E[Evaluate]
  E -->|pass| R[Return]
  E -->|revise| G
```

Source: `evaluator-optimizer-pattern`

Self-Improvement

Self-improvement uses feedback from prior runs to improve future runs. In production, this should usually mean improving prompts, tools, tests, retrieval, policies, or skills through reviewed changes, not letting an agent silently rewrite itself.

Source: `reflection-and-self-improvement-pattern`

Self-Healing Workflows

Self-healing workflows detect failed steps and recover through retry, fallback, re-planning, or escalation.

The recovery policy should be explicit. Not every failure should trigger another model call.

Source: `self-healing-workflow-agent-pattern`

Memory-Augmented Agent

Memory-augmented agents store and retrieve information across turns or sessions.

Separate memory types: short-term conversation state, long-term episodic memory, semantic recall, and structured working memory.

Source: `memory-augmented-agent-pattern`

Long-Term Episodic Memory

Long-term episodic memory stores events: what happened, when, who was involved, and why it mattered.

Use it for assistants that need continuity across sessions. Retrieve memories by relevance, recency, and user or project scope.

Source: `long-term-episodic-memory-agent-pattern`

Semantic Recall and RAG

Semantic recall retrieves relevant material by meaning rather than exact keywords. RAG injects retrieved material into context before generation.

Use it when the agent must answer from a changing or large knowledge base.

Source: `context-engineering-pattern`

Working Memory

Working memory is structured state the agent can update and consult during a task: user preferences, current plan, constraints, entities, and open questions.

Unlike raw chat history, working memory is compact and typed.

Related source: `goals-and-state-pattern`

Knowledge-Bound Agents

Knowledge-bound agents must ground answers and actions in approved sources. They are useful for compliance, support, technical docs, and internal operations.

Design the boundary explicitly: allowed sources, citation rules, freshness requirements, refusal behavior, and escalation paths.

Related source: `compliance-policy-enforcer-agent`

Skills

Skills package procedural knowledge as discoverable folders of instructions, references, scripts, templates, and assets.

They are useful when the agent needs more than a tool call: it needs a repeatable procedure.

Source: `skills-pattern`

MCP-first Tool Use

MCP-first tool use separates tool capability from agent logic.

Agents discover manifests, validate inputs, invoke tools, and handle structured outputs through a stable boundary.

```
sequenceDiagram
  participant Agent
  participant MCPServer
  Agent->>MCPServer: Read manifest
  Agent->>Agent: Validate input
  Agent->>MCPServer: Invoke tool
  MCPServer-->>Agent: Structured result
```

Use this pattern when tools evolve independently from agents or must be shared across runtimes.

Source: `modern-tool-use-pattern`

A2A Agent Interoperability

A2A lets agents collaborate across process, runtime, team, or vendor boundaries.

The core idea is that a remote agent should be discoverable and callable through a protocol contract, not embedded as local implementation detail.

```
sequenceDiagram
  participant ClientAgent
  participant AgentCard
  participant RemoteAgent
  ClientAgent->>AgentCard: Fetch capabilities
  ClientAgent->>RemoteAgent: Send task request
  RemoteAgent-->>ClientAgent: Progress, result, refusal, or error
```

Use A2A when agents need interoperability, asynchronous progress, refusals, cancellation, and task identity.

Source: `agent-to-agent-communication-pattern`

Secure Agent Communication

Secure communication protects messages between agents with authentication, integrity, confidentiality, and policy checks.

Use it when agents cross trust boundaries, handle private data, or trigger external side effects.

Source: `secure-agent-communication-pattern`

Human Approval Gates

Human approval gates pause execution before sensitive, expensive, destructive, or externally visible actions.

The workflow should preserve state while waiting and record who approved what, when, and with which context.

Source: `human-in-the-loop-approval-agent`

Task Delegation

Task delegation assigns bounded subtasks to specialized workers and combines their outputs.

The manager should define expected outputs, constraints, and acceptance criteria for each worker.

Source: `task-delegation-pattern`

Supervisor / Worker

Supervisor/Worker is a stricter form of delegation. The supervisor owns the goal, task state, routing, and quality gate. Workers perform bounded work and return structured results.

Use this pattern when independent specialists are useful but centralized control is still required.

Related source: `hierarchical-agent-pattern`

Debate and Consensus

Debate and consensus patterns use multiple independent proposals, critiques, votes, or rankings before producing a final answer.

They are useful when diversity of reasoning improves quality, but they need clear aggregation rules.

Source: `consensus-seeking-multi-agent-system-pattern`

Parallel Agents

Parallel agents run independent work concurrently, then merge results.

Use this for search, research, independent review, candidate generation, or latency-sensitive fan-out/fan-in tasks.

Related source: `multi-agent-collaboration-pattern`

CrewAI Flows and Crews

CrewAI Flows own state and execution order. Crews group specialized agents that collaborate on delegated work inside the flow.

Source: `crewai-flows-and-crews-pattern`

Durable Workflows

Durable workflows make agentic systems resumable and auditable. The workflow owns retries, checkpoints, approvals, compensation, and long-running state.

Source: `durable-workflow-pattern`

Observability and Evals

Observability records what happened. Evals decide whether behavior is good enough.

Production agents need traces, model inputs, tool calls, costs, latency, evaluator scores, and regression datasets.

Source: `observability-and-evals-pattern`

Policy Enforcement

Policy enforcement constrains what the agent may say or do. It covers tool permissions, data access, business rules, safety rules, and escalation.

Source: `compliance-policy-enforcer-agent`

Event-Triggered Agents

Event-triggered agents run in response to webhooks, queues, schedules, or domain events.

They should be idempotent, observable, and bounded because no human may be watching the interaction live.

Source: `event-triggered-agent-pattern`

Mastra Runtime

Mastra is a TypeScript runtime pattern for applications that need agents, workflows, tools, memory, evals, and observability in one framework.

Use it to host patterns, not to replace architectural thinking.

Source: `mastra-runtime-pattern`

Deprecated / Historical Patterns

Deprecated patterns are preserved in the repository but removed from the active learning path.

They are either speculative, too broad, duplicated by stronger chapters, or currently too thin to stand as active patterns.

- Agent Marketplace: speculative compared with A2A, routing, and orchestration.
- Agent Swarm: folded into Parallel Agents, search, and optimization.
- Hybrid Agent: too broad to teach as a standalone pattern.
- Meta-Cognitive Agent: folded into Reflection, Evaluator-Optimizer, and Self-Improvement.
- Recursive Agent: folded into Planning, Goals and State, and Task Delegation.
- Distributed Agent: replaced by A2A, durable workflows, and protocol-first composition.
- API Integration Copilot: archived as an applied placeholder.
- Data Pipeline Orchestrator Agent: archived as an applied placeholder.
- Multi-Modal Tool-Using Agent: archived until it has a developed multimodal implementation.

Source archive: `deprecated`